

Classical Conditioning and Reinforcement Learning

Euntaek Lee

Seoul National University

May 12, 2023

Table of contents

1. Classical conditioning
2. Static Action Choice
3. Sequential Action Choice
4. Generalized sequential Action choice

Pavlovian experiment

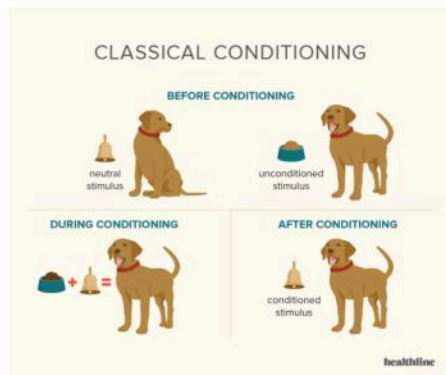


Figure: Pavlovian experiment¹

¹<https://www.healthline.com/health/classical-conditioning>

Rescorla-Wagner rule

Assumption

$$(\text{expected reward } v) = (\text{weight } w) \times (\text{stimulus } u)$$

Rescorla-Wagner rule

$$w \rightarrow w + \varepsilon \delta u, \quad \delta = r - v$$

for given reward r .

Numeric result

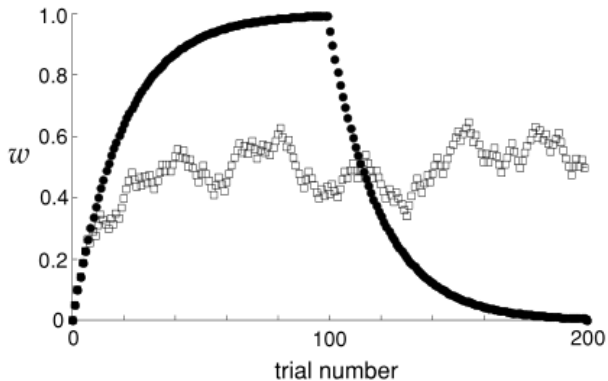


Figure: Rescorla-Wagner model

Rescorla-Wagner rule

For multiple types of stimuli, we consider multidimensional generalization of R-W rule by

$$v = \mathbf{w} \cdot \mathbf{u}, \quad \delta = r - v, \quad \mathbf{w} \rightarrow \mathbf{w} + \varepsilon \delta \mathbf{u}$$

for given reward r . This can be used for describing some classical conditioning paradigms.

Conditioning paradigms

Paradigm	Pre-train	Train	Result
Pavlovian		$s \rightarrow r$	$s \rightarrow (r)$
Extinction	$s \rightarrow r$	$s \rightarrow \cdot$	$s \rightarrow 0$
Partial		$s \rightarrow r, s \rightarrow \cdot$	$s \rightarrow \alpha(r)$
Blocking	$s_1 \rightarrow r$	$s_1 + s_2 \rightarrow r$	$s_1 \rightarrow (r), s_2 \rightarrow 0$
Inhibitory		$s_1 + s_2 \rightarrow \cdot, s_1 \rightarrow r$	$s_1 \rightarrow (r), s_2 \rightarrow -(r)$
Overshadow		$s_1 + s_2 \rightarrow r$	$s_1 \rightarrow \alpha_1(r), s_2 \rightarrow \alpha_2(r)$
Secondary	$s_1 \rightarrow r$	$s_2 \rightarrow s_1$	$s_2 \rightarrow (r)$

Table: Classical conditioning paradigms

Blocking

- ▶ Pre-trained : $s_1 \rightarrow r$,
- ▶ Train : $s_1 + s_2 \rightarrow r$,
- ▶ Result : $s_1 \rightarrow (r)$, $s_2 \rightarrow 0$

$$v = \mathbf{w} \cdot \mathbf{u}, \quad \delta = r - v, \quad \mathbf{w} \rightarrow \mathbf{w} + \epsilon \delta \mathbf{u}$$

Secondary

- ▶ Pre-train : $s_1 \rightarrow r$
- ▶ Train : $s_2 \rightarrow s_1$
- ▶ Result : $s_2 \rightarrow (r)$

Temporal Difference Learning

Here we consider

$$u = u(t), v = v(t), r = r(t), \quad 0 \leq t \leq T.$$

We suggest

$$\left\langle \sum_{\tau=0}^{T-t} r(t+\tau) \right\rangle = v(t) \approx \sum_{\tau=0}^t w(\tau) u(t-\tau)$$

Temporal Difference Learning

and update $w(t)$ by

$$w(\tau) \rightarrow w(\tau) + \varepsilon \delta(t) u(t - \tau)$$

with

$$\begin{aligned} \delta(t) &:= \sum_{\tau=0}^{T-t} r(t + \tau) - v(t). \\ &= (\text{Full potential reward}) - (\text{predicted reward}) \end{aligned}$$

Temporal Difference Learning

and update $w(t)$ by

$$w(\tau) \rightarrow w(\tau) + \varepsilon \delta(t) u(t - \tau)$$

with

$$\delta(t) := \sum_{\tau=0}^{T-t} r(t + \tau) - v(t).$$

By (9.8)-(9.10) in textbook, we can simplify $\delta(t)$ by

$$\delta(t) = r(t) + v(t + 1) - v(t).$$

Numeric simulation

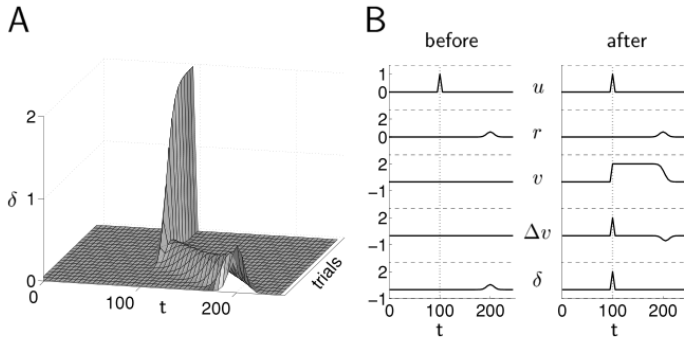


Figure: Learning to predict a reward

Temporal Difference Learning

Remark

We can explain the secondary conditioning with TDL.

Experiment with bee

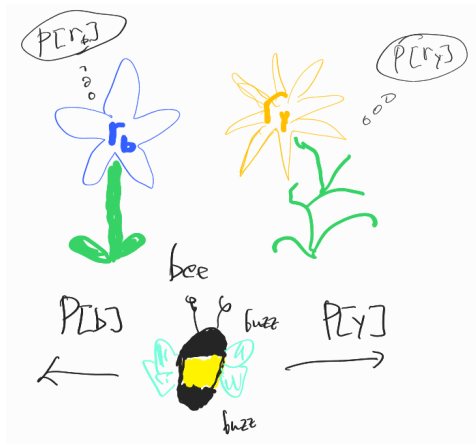


Figure: Experiment with bee

Policy of bee

Assume the bee chooses blue and yellow flowers with probability

$$P[b] = \frac{\exp(\beta m_b)}{\exp(\beta m_b) + \exp(\beta m_y)}, \quad P[y] = \frac{\exp(\beta m_y)}{\exp(\beta m_b) + \exp(\beta m_y)}.$$

We can represent it by

$$P[b] = \sigma(\beta(m_y - m_b)), \quad P[y] = \sigma(\beta(m_b - m_y))$$

for standard sigmoid function

$$\sigma(m) = \frac{1}{1 + \exp(-m)}.$$

Question

How does a bee learn optimal parameter?

The indirect actor

Here, “Indirect” means learn

$$m_b = \langle r_b \rangle, \quad m_y = \langle r_y \rangle$$

without using probability by δ -method

$$\begin{cases} m_b \rightarrow m_b + \epsilon \delta, & \delta = r_b - m_b & \text{lands b} \\ m_y \rightarrow m_y + \epsilon \delta, & \delta = r_y - m_y & \text{lands y} \end{cases}$$

The direct actor

Here, “direct” means learn

$$m_b = \langle r_b \rangle, \quad m_y = \langle r_y \rangle$$

with using probability with

$$\begin{cases} m_b \rightarrow m_b + \epsilon (\delta_{ab} - P[b]) (r_a - \bar{r}) \\ m_y \rightarrow m_y + \epsilon (\delta_{ay} - P[y]) (r_a - \bar{r}) \end{cases}$$

where a is the action selected, $\bar{r}$ is a parameter.

Derivation

We can derive

$$\frac{\partial \langle r \rangle}{\partial m_b} = \beta P[b] ((1 - P[b]) (\langle r_b \rangle - \bar{r})) \\ - \beta P[y] (P[b] (\langle r_y \rangle - \bar{r}))$$

and if we update m_b by

$$m_b \rightarrow \begin{cases} m_b + ((1 - P[b]) (\langle r_b \rangle - \bar{r})) & (b) \\ m_b - (P[b] (\langle r_y \rangle - \bar{r})) & (y), \end{cases}$$

with this process, the expectation of (m_b, m_y) is **the direction of gradient** $\left(\frac{\partial \langle r \rangle}{\partial m_b}, \frac{\partial \langle r \rangle}{\partial m_y} \right)$.

Comparison

- ▶ The indirect method has better performance.
- ▶ The direct actor will be useful later.

Generalization

For multivariable case, the softmax policy is

$$P[a] = \frac{\exp(\beta m_a)}{\sum_{a'=1}^{N_a} \exp(\beta m_{a'})}, \quad a = \{1, 2, \dots, N_a\},$$

the direct actor learning rule is

$$m_{a'} \rightarrow m_{a'} + \epsilon (\delta_{aa'} - P[a']) (r_a - \bar{r}), \quad a' \in 1, 2, \dots, N_a$$

for chosen action a .

The maze task

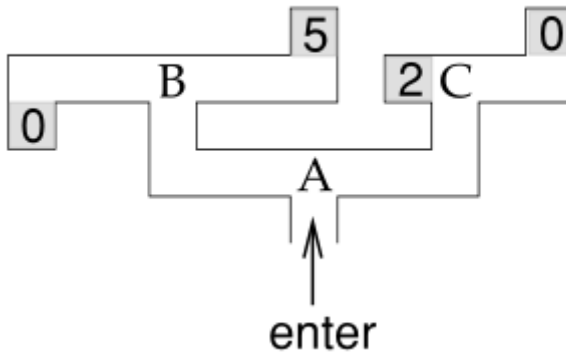


Figure: The maze task

Actor-critic learning

- ▶ $\mathbf{m} : \{A, B, C\} \rightarrow \mathbb{R}^2$ for $u \in \{A, B, C\}$,
 $\mathbf{m}(u)_1 = \sigma(\beta(m_{u,L} - m_{u,R}))$, $\mathbf{m}(u)_2 = \sigma(\beta(m_{u,L} - m_{u,R}))$
- ▶ $r_a(u) : \{L, R\} \times \{A, B, C\} \rightarrow \{0, 2, 5\}$, The immediate reward.
- ▶ $w(u) = v(u) : \{A, B, C\} \rightarrow \mathbb{R}$, the weight and predicted reward.

Policy evaluation

- ▶ To learn : $v(B) = 2.5, v(C) = 1, v(A) = 1.75$.
- ▶ Learning rule:

$$w(u) \rightarrow w(u) + \varepsilon \delta, \quad \delta = r_a(u) + v(u') - v(u)$$

if the rat chooses the action

$$a \in \{L, R\}, \quad u \xrightarrow{a} u'.$$

Numeric simulation

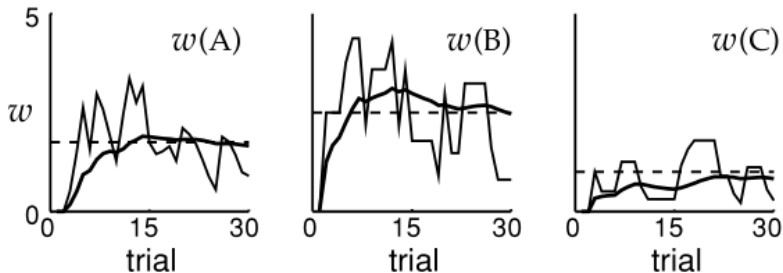


Figure: Policy evaluation

Policy improvement

Lending direct actor policy gives

$$m_{a'} \rightarrow m_{a'} + \epsilon (\delta_{aa'} - P[a'; u]) (r_a - \bar{r})$$

and we obtain

$$m_{a'} \rightarrow m_{a'} + \epsilon (\delta_{aa'} - P[a'; u]) \delta$$

where

$$\delta = r_a(u) + v(u') - v(u).$$

Numeric simulation

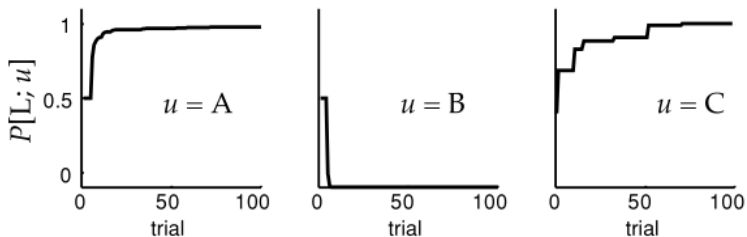


Figure: Actor-critic learning

Multidimensional A-C

- ▶ $\mathcal{U}$: the set of location.
- ▶ $\mathbf{m} : \mathcal{U} \rightarrow \mathbb{R}^{|\mathcal{A}|}$, output is softmax probability density.
- ▶ $r_a(u) : \mathcal{A} \times \mathcal{U} \rightarrow \mathcal{R}$, The immediate reward.
- ▶ $\mathbf{u} : \mathcal{U} \rightarrow \mathbb{R}^d$, the state vector.
- ▶ $\mathbf{w} : \mathcal{U} \rightarrow \mathbb{R}^d$, the weight.
- ▶ $v = \mathbf{w} \cdot \mathbf{u}(u)$, the predicted value.

Learning rule

- ▶ Step 1 : Policy evaluation, improve $\mathbf{w}$ by

$$\mathbf{w} \rightarrow \mathbf{w} + \varepsilon \delta \mathbf{u}(u), \quad \delta = r_a(u) + v(u') - v(u).$$

- ▶ Step 2 : Policy improvement, improve

$$M_{a'b} \rightarrow M_{a'b} + \varepsilon (\delta_{aa'} - P[a'; u]) \delta u_b(u).$$

For

$$\mathbf{m} = \mathbf{M} \cdot \mathbf{u}(u) \text{ for } \mathbf{M} \in M_{|\mathcal{A}| \times d}(\mathbb{R}).$$

Reward-Punishment

- ▶ Step 1 : Policy evaluation, improve $\mathbf{w}$ by

$$\mathbf{w} \rightarrow \mathbf{w} + \varepsilon \delta \mathbf{u}(u), \quad \delta = r_a(u) + \gamma v(u') - v(u).$$

- ▶ Step 2 : Policy improvement, improve

$$M_{a'b} \rightarrow M_{a'b} + \varepsilon (\delta_{aa'} - P[a'; u]) \delta u_b(u).$$

Water maze task



Figure: The maze task

Numeric simulation

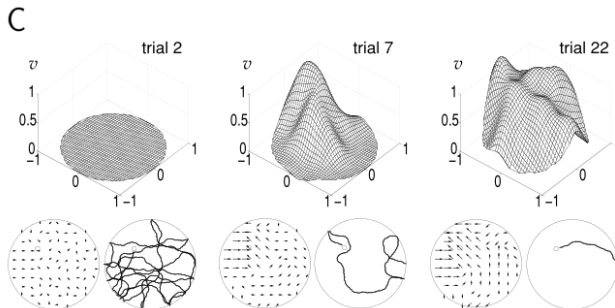


Figure: Simulation of A-C procedure

Thank you for listening!